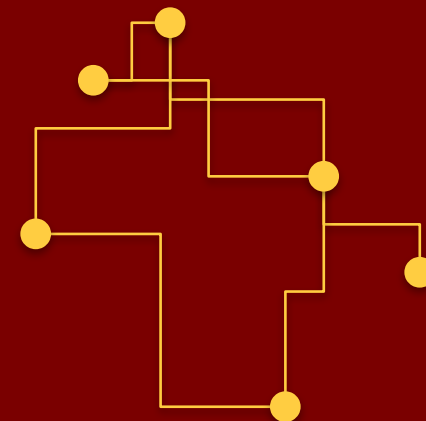


TEAM INTELLIAI

IntelliAI

Intelligent MQ Topology Simplification & Modernization

Part 1 — A 10-agent LangGraph pipeline that turns a legacy MQ estate into a compliant, automation-ready target state



10

AI agents

259 → 433

queue managers

100 → 45

complexity score

~30s

end-to-end run

THE PROBLEM

A legacy MQ estate is a tangled, fragile mesh

Real dataset profiled by the pipeline — 12,696 raw rows of production-scale MQ inventory.



5.16

Coupling index — apps wired to 5+ QMs on average. No 1:1 ownership.



310

Multi-QM app violations — the core constraint breached across the estate.



53

Orphan queue managers — running, costing, connected to nothing.



21

Routing cycles — A→B→C→A loops risking message storms.



74

Disconnected components — fragmented islands, diameter of 79 hops.



100/100

As-is complexity — the topology pins the scale at maximum.

Hybrid intelligence: rules guarantee, LLM reasons

The design decision that makes this trustworthy for an enterprise MQ estate.

Rule Engine

Deterministic · Zero-tolerance

- 1:1 app-to-QM assignment
- Deterministic channel naming — FROM_QM.TO_QM
- XMITQ + routing pattern provisioning
- Runs in milliseconds on 438 apps — 100% compliance

LLM Agents

Tachyon → Gemini 2.0 Flash · Reasoning

- PCI / payment-zone isolation decisions
- Cluster analysis & bridge-app placement
- Natural-language human feedback interpretation
- Every proposal validated back through the rules

✓ **Self-correcting:** if the LLM proposes an assignment that breaks a constraint, the rule engine rejects it with a logged explanation. **Full rule-only fallback** produces a valid target state even with no LLM available.

Ten agents, one LangGraph state machine

1

Supervisor

Session init, file validation, routing

2

Sanitiser

Clean, dedupe, normalise → 4 logical tables

3

Researcher

As-is graph • Louvain • centrality • entropy

4

Analyst

6-factor complexity score of as-is

5

Architect

Target design — rules + LLM cluster analysis

6

Optimizer

Reachability prune + weighted MST reduction

7

Tester

8 hard constraints • 0 critical = PASS

9

Provisioner

Per-QM MQSC scripts + target CSVs

10

Migration Planner

2,605 ordered steps • 4 phases • rollback

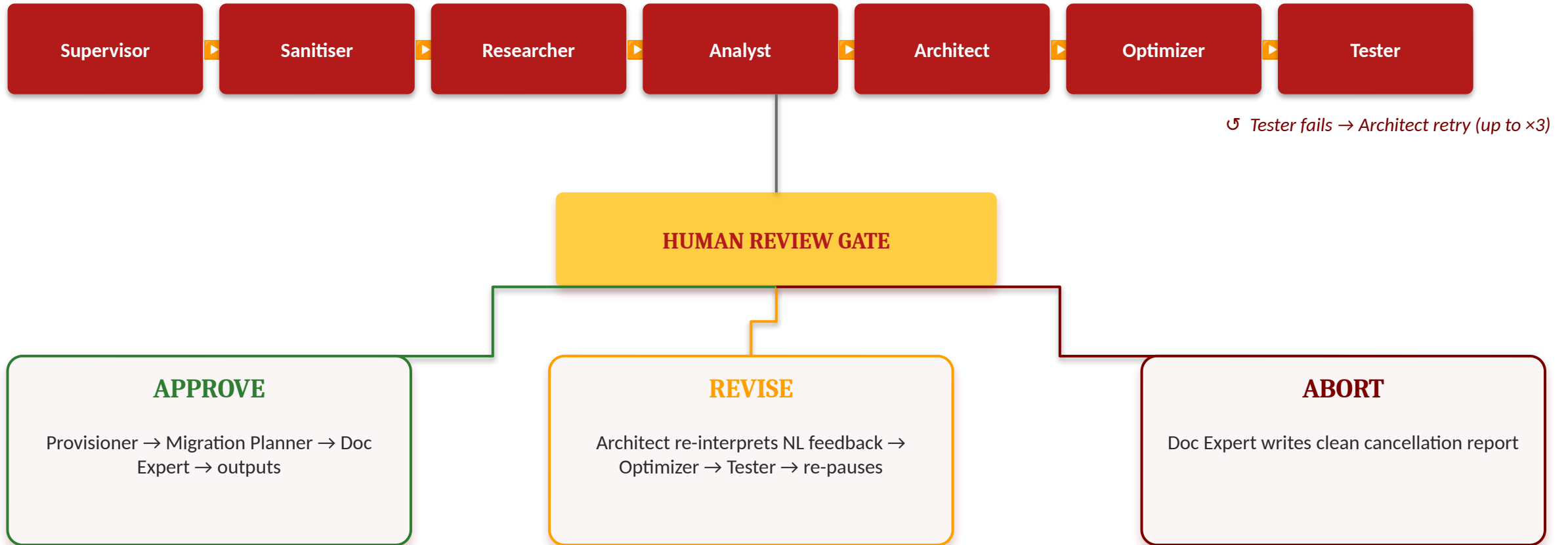
11

Doc Expert

ADRs, exec summary, regression + insights

Human Review Gate — pipeline pauses after Tester • Approve / Revise / Abort • "Ask the Architect" chat

Phase 1 designs. Human decides. Phase 2 executes.



The Revise path skips Supervisor→Analyst — input is unchanged, so the Architect applies targeted deltas to the existing target graph instead of redesigning from scratch.

Inside the Architect: three phases, self-correcting



PHASE A

Rules Baseline

Deterministic engine assigns all 438 apps to dedicated QMs (438→438). Mathematical guarantee of 1-QM-per-app. No LLM.



PHASE B

LLM Cluster Analysis

~4,600-token cluster prompt to Gemini. Isolates PCI bridge apps (QM_PCI_*), separates payment-critical workloads (QM_PAY_CRIT_*) to shrink blast radius.



PHASE C

Validated Assembly

10–12 reassignments applied, 0 rejected. Every LLM move re-checked by the rule engine — INCORRECT ones bounced with a logged reason. ADRs generated.



Revision mode — same agent, different job

On a Revise decision the Architect parses natural-language feedback into structured directives — *aggressive*, *fanout_cap*, *channel_prune_pct*, *consolidate_qm_pct*, *protect_qms* — with HIGH/MEDIUM/LOW confidence, then applies targeted deltas rather than redesigning. Demonstrated: 446→433 QMs, 22→9 warnings, score 48.3→45.4.

Guardrails: the LLM is never trusted blindly

1 Rules-validated output

Every LLM assignment is re-checked by the deterministic engine. A move that breaks a constraint is rejected with a logged reason — the LLM cannot corrupt the target state.

2 Strict JSON schema

Responses forced to json_object mode against a fixed schema (assignments, channels, ADRs). Missing keys are detected and the call is retried or discarded.

3 Self-correcting retry

On invalid JSON the parser feeds the exact error back into the prompt so the model repairs its own output — bounded to 2 retries, never an infinite loop.

4 Circuit breaker

A 429 or rate-limit trips a 60-second breaker: the pipeline stops calling the LLM and runs on rules, so one throttle never stalls the whole run.

5 Full rule-only fallback

No credentials, timeout, or oversized payload (413) → immediate graceful fallback. The pipeline always emits a valid, compliant target state.

6 Grounded, no free invention

Prompts instruct the model to reference ONLY entity names present in the input data — the rule engine then discards any QM or app the LLM invents.

Tight prompts, hardened inputs, minimal tokens

Prompt design & injection defense

- **Role-scoped system prompts** — 12 narrow roles, each with a fixed job, temp and token cap — no open-ended "do anything" agent.
- **Low temperature (0.1)** — Design calls run near-deterministic; creativity is off where correctness matters.
- **Data / instruction separation** — Topology facts go in the user turn; the immutable contract lives in the system prompt the data can't overwrite.
- **Reference-only grounding** — Model may cite only entities in the payload; rules discard invented names — blocks data-driven injection.
- **Output-side enforcement** — Even a hijacked reply must pass schema + rules validation, so a malicious instruction can't reach the target state.

Token economics

13,000 rows → ~3K tokens

The estate is summarised into structured text before it ever reaches the model — we never paste raw CSV.

- **Summarise, don't dump** — 13K rows compressed to a ~3–6K-token structured brief regardless of estate size.
- **Cap the payload** — Channel lists truncated to a fixed budget; oversized prompts fall back to rules, never blow the context window.
- **JSON-only responses** — No prose padding — the model returns compact structured output, cutting output tokens.
- **Rules do the bulk work** — 438 deterministic assignments cost 0 tokens; the LLM is called only for the judgment calls.

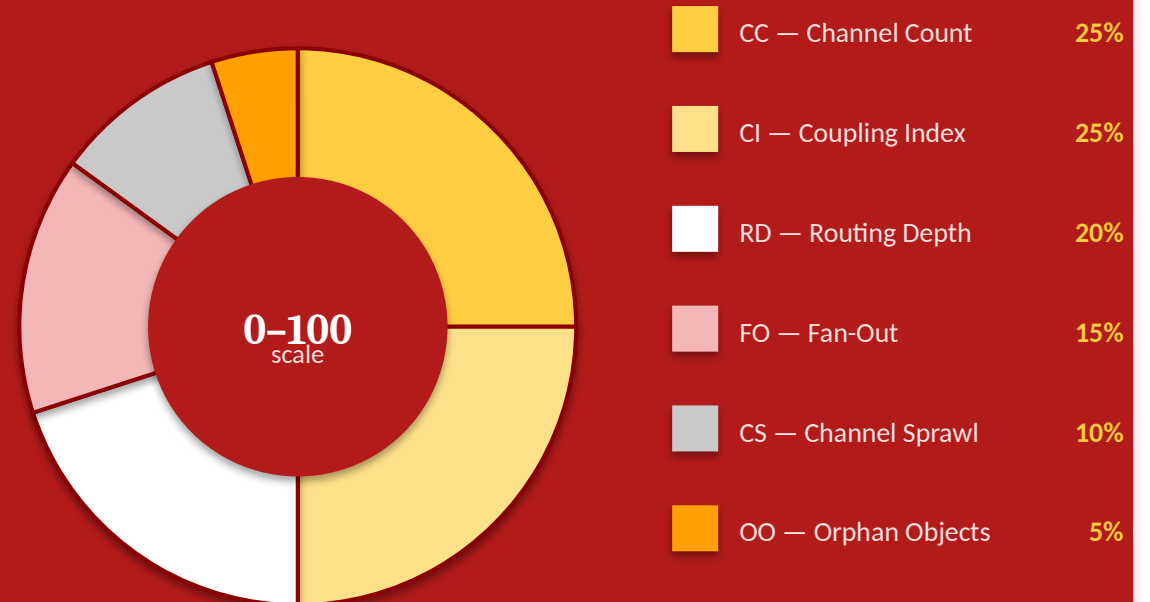
Real graph analytics, not vibes

Graph algorithms driving decisions

- **Louvain community detection** — auto-discovers QM clusters — no k needed
- **Betweenness centrality** — flags true SPOFs, not just busy nodes
- **Shannon entropy** — quantifies hub-and-spoke fragility
- **Minimum spanning tree (Kruskal)** — provably minimal channel set to stay connected
- **Reachability + MST pruning** — removes dead + redundant channels
- **Kernighan-Lin bisection** — finds clean regional split points

6-factor complexity score

Each factor normalised 0–100 vs baseline, weighted sum on a 0–100 scale.



As-is → target: measured, not asserted

COMPLEXITY

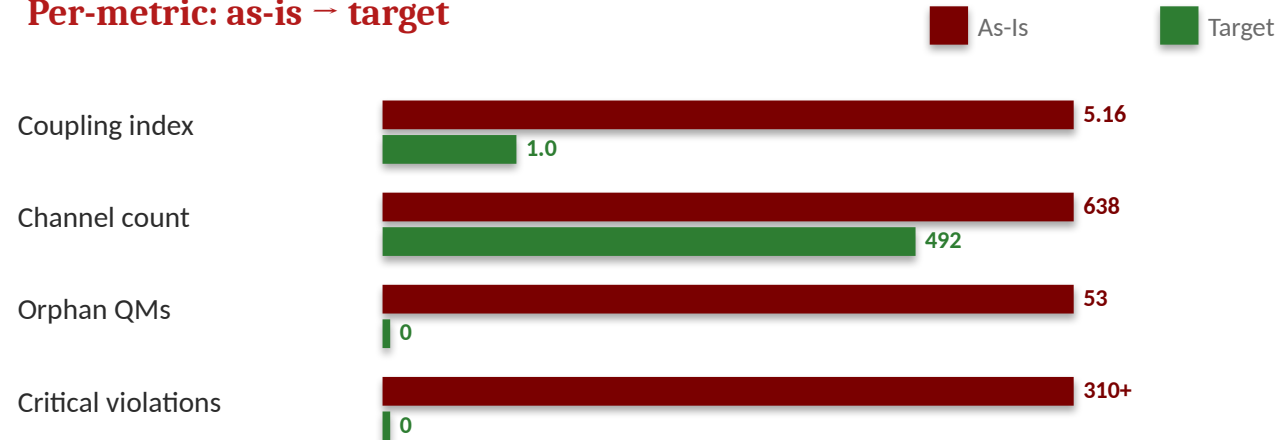
54.6%

reduction

100.0 → 45.4

on the 6-factor scale

Per-metric: as-is → target



80.6%

Coupling index 5.16 → 1.0

100%

Orphan QMs 53 → 0

100%

Critical violations 310+ → 0

22.9%

Channel count 638 → 492

Manual vs IntelliAI: the effort collapse

We can't quote a licence-cost figure, but the human-effort delta on this single 259-QM estate is the story.

TASK**MANUAL TODAY****WITH INTELLIAI****Profile estate & detect violations**

Days of manual spreadsheet + query work

Seconds — 74 components, 310 violations, SPOFs auto-found

Design compliant target topology

Senior architect, weeks of whiteboarding

Rules + LLM, validated, in one ~30s run

Write MQSC provisioning scripts

Hand-authored per QM, error-prone

3,750 commands across 433 QMs, dependency-ordered

Build phased migration + rollback

Multi-week planning exercise

2,605 ordered steps, 4 phases, rollback per step

Documentation (ADRs, exec summary, tests)

Often skipped under deadline

Auto-generated, enterprise-grade, every run

Future roadmap: Day-2 lifecycle management

Today the pipeline transforms an estate once. Next: keep it optimised as the estate lives and breathes — the same 10 agents applied incrementally.



Add App

New application enters the estate. The agent picks optimal QM placement from producer/consumer profile, traffic, and compliance zone — then provisions QM, channels, queues and re-validates via Tester.



Reroute / Reconnect

App changes upstream or downstream partners. The agent computes the channel delta, checks for new SPOFs or compliance-boundary breaches, and emits a REROUTE plan with rollback.



Remove App

Application decommissioned. The agent traces every dependent channel, queue and XMITQ, generates CLEANUP MQSC, and evaluates the orphaned QM for consolidation or decommission.

🔄 **One pipeline, two modes:** bulk transformation for the initial modernization, incremental lifecycle ops thereafter. The topology never drifts back into a tangled mesh.

NOW THE DEMO

Let's watch the pipeline run

Upload a raw MQ dataset → 10 agents analyse, design, validate → pause at the human gate → approve and generate every deliverable.

Upload

raw CSV, one file

Trace

every agent step live

Topology

as-is vs target

Review

approve / revise / ask